

Procesor superskalarny i out-of-order

ASK 2026

Instytut Informatyki
Uniwersytet Wrocławski

10 kwietnia 2026

Dzisiaj omówimy:

- prosty schemat procesora jednocyklowego
- procesor potokowy (pipelining) i hazardy
- procesor superskalarny i out-of-order

Nie wszystkie fragmenty będą dotyczyły się architektury x86_64.

Procesor jednocyklowy

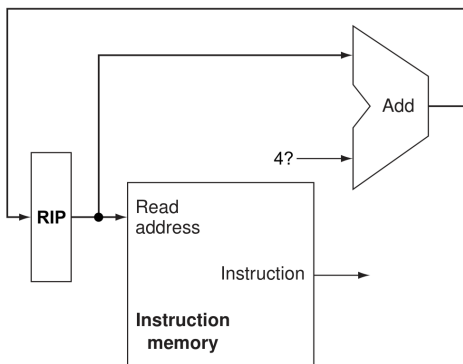
Do budowy procesora będziemy potrzebować:

- jednostki arytmetyczno-logicznej
- rejestrów
- pamięci – do przechowywania danych i instrukcji

Będziemy kolejno wykonywać fazy:

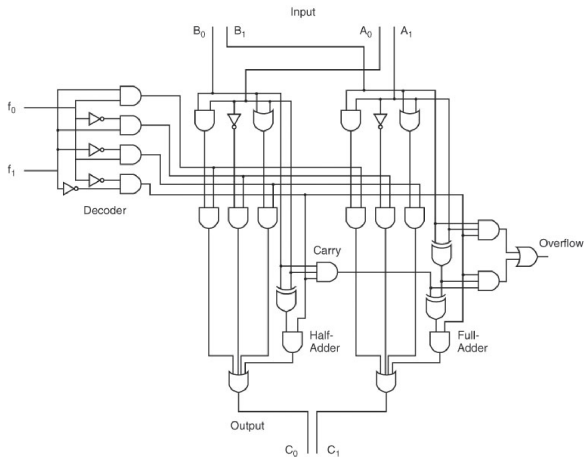
- wczytania instrukcji
- pobrania danych z rejestrów
- obliczenia arytmetyczno-logiczne
- odczyt/zapis do pamięci
- zapis do rejestrów

Układ czytający instrukcje

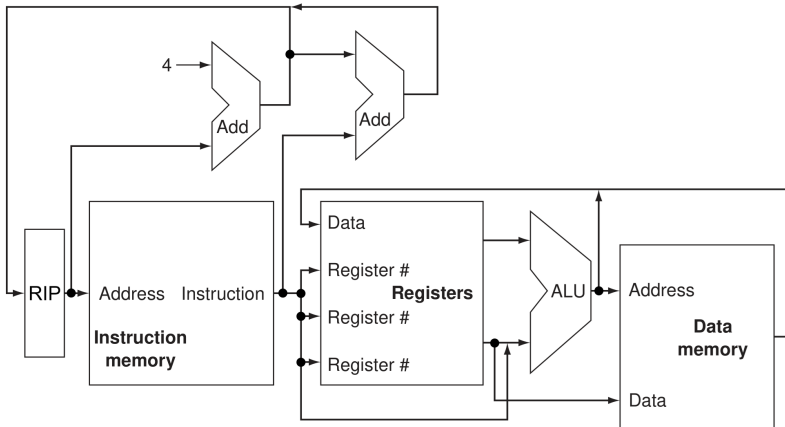


W architekturze MIPS każda instrukcja ma rozmiar 4 bajtów.

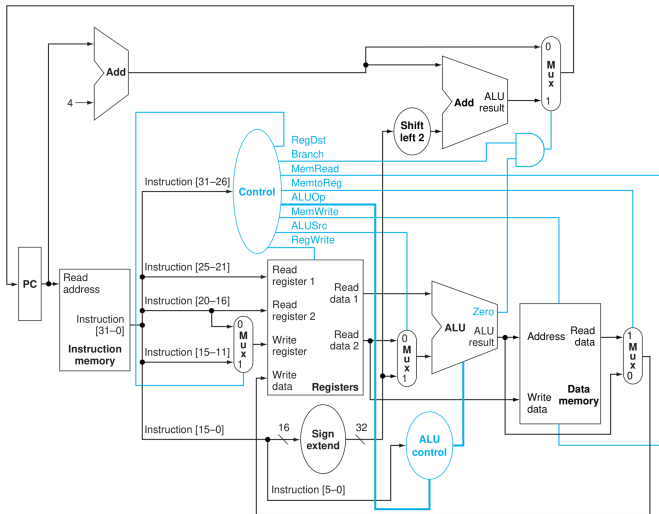
Jednostka arytmetyczno-logiczna (ALU)



Prosty schemat procesora jednocyklowego



Trochę dokładniejszy schemat procesora MIPS



Idea procesora potokowego

Każda instrukcja musi przejść przez wszystkie części układu. Wiele z nich jest przez większość czasu niewykorzystanych.

Pomysł – będziemy wykonywali kilka instrukcji jednocześnie.

Idea procesora potokowego

Każda instrukcja musi przejść przez wszystkie części układu. Wiele z nich jest przez większość czasu niewykorzystanych.

Pomysł – będziemy wykonywali kilka instrukcji jednocześnie.

Podzielimy wykonanie instrukcji na 5 kolejnych faz:

1. IF: Instruction Fetch – pobranie instrukcji z pamięci
2. ID: Instruction Decode – pobranie wartości z rejestrów
3. EX: Execute – wykonanie operacji arytmetyczno-logicznych
4. MEM: Memory access – odczyty z/zapisy do pamięci
5. WB: Write Back – zapis wartości do rejestrów

Nie można wykonać na przykład `add (%rax), %rdx`.

Idea procesora potokowego

Przyjmijmy następujące założenia:

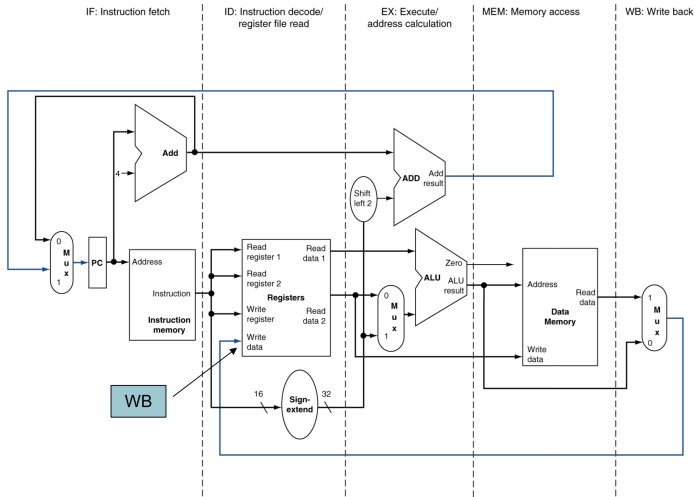
- IF – korzysta z pamięci instrukcji, trwa 200ps
- ID – korzysta z rejestrów, trwa 100ps
- EX – korzysta z ALU, trwa 200ps
- MEM – korzysta z pamięci danych, trwa 200ps
- WB – korzysta z rejestrów, trwa 100ps

Jedna instrukcja zajmuje procesor sumarycznie przez 800ps.

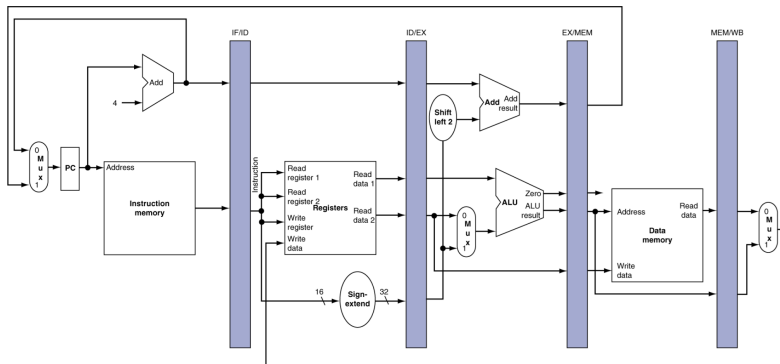
Jedynie ID i WB potrzebują tej samej jednostki (dostępu do rejestrów) w tym samym czasie.

Zauważmy też, że jedna instrukcja zajmuje każdą jednostkę sumarycznie przez 200ps. Spróbujmy to wykorzystać.

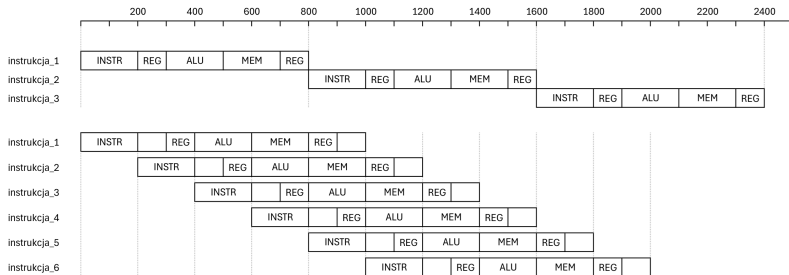
MIPS – podział na fazy



MIPS – podział na fazy



Pipelining



Przetwarzanie potokowe

Czy coś tu może pójść nie tak?

Przetwarzanie potokowe

Czy coś tu może pójść nie tak?

Hazardy:

- hazardy strukturalne – dwie instrukcje chcą skorzystać z tej samej jednostki w tym samym momencie

Przetwarzanie potokowe

Czy coś tu może pójść nie tak?

Hazardy:

- hazardy strukturalne – dwie instrukcje chcą skorzystać z tej samej jednostki w tym samym momencie (**resolved**)
- hazardy danych – jedna instrukcja chce skorzystać z danych, których inna jeszcze nie zdążyła zapisać, na przykład:

```
add %rax, %rdx
sub %rdx, %rcx
```

tutaj odczyt z rejestru `%rdx` w instrukcji `sub` musi zostać wykonany później niż zapis do tego rejestru w operacji `add`

Przetwarzanie potokowe

Czy coś tu może pójść nie tak?

Hazardy:

- hazardy strukturalne – dwie instrukcje chcą skorzystać z tej samej jednostki w tym samym momencie (**resolved**)
- hazardy danych – jedna instrukcja chce skorzystać z danych, których inna jeszcze nie zdążyła zapisać, na przykład:

```
add %rax, %rdx
sub %rdx, %rcx
```

tutaj odczyt z rejestru `%rdx` w instrukcji `sub` musi zostać wykonany później niż zapis do tego rejestru w operacji `add` (**pending**)

- hazardy sterowania – w przypadku skoku nie wiemy którą instrukcję wykonać jako kolejną

Przetwarzanie potokowe

Czy coś tu może pójść nie tak?

Hazardy:

- hazardy strukturalne – dwie instrukcje chcą skorzystać z tej samej jednostki w tym samym momencie (**resolved**)
- hazardy danych – jedna instrukcja chce skorzystać z danych, których inna jeszcze nie zdążyła zapisać, na przykład:

```
add %rax, %rdx
sub %rdx, %rcx
```

tutaj odczyt z rejestru `%rdx` w instrukcji `sub` musi zostać wykonany później niż zapis do tego rejestru w operacji `add` (**pending**)

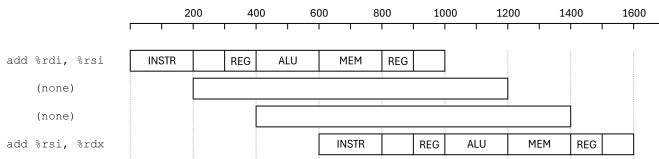
- hazardy sterowania – w przypadku skoku nie wiemy którą instrukcję wykonać jako kolejną (**backlog**)

Hazardy danych

```
add %rdi, %rsi
add %rsi, %rdx
```

Pomysły na rozwiązanie:

- opóźnić drugą instrukcję

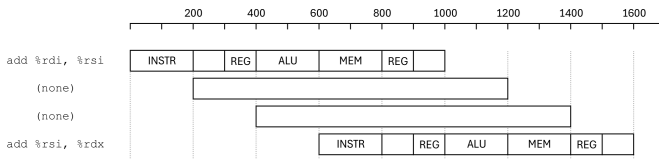


Hazardy danych

```
add %rdi, %rsi
add %rsi, %rdx
```

Pomysły na rozwiązanie:

- opóźnić drugą instrukcję



→ niepotrzebne opóźnienie wykonywania

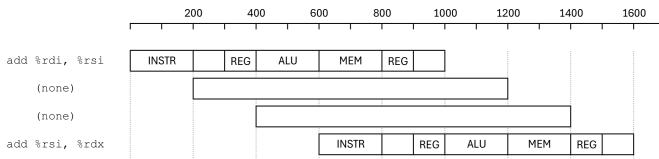
- jakoś przekazać dane od razu jak już zostaną wyliczone

Hazardy danych

```
add %rdi, %rsi  
add %rsi, %rdx
```

Pomysły na rozwiązanie:

- opóźnić drugą instrukcję



→ niepotrzebne opóźnienie wykonywania

- jakoś przekazać dane od razu jak już zostaną wyliczone
- przekładać instrukcje, żeby zmniejszyć liczbę hazardów

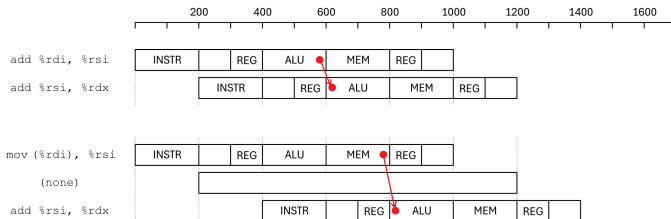
Hazardy danych

```
add %rdi, %rsi
add %rsi, %rdx
```

Zauważmy, że pierwsza instrukcja zna wynik już w momencie zakończenia obliczeń w ALU. Za to druga potrzebuje go bezpośrednio przed wejściem do ALU.

Można zastosować tzw. *forwarding*, czyli przekazywanie danych do kolejnej instrukcji zanim zostaną zapisane w rejestrach.

Sytuacja jest trudniejsza gdy dana wartość musi być odczytana z pamięci, wtedy konieczne jest dodanie opóźnienia o jeden cykl.



Hazardy danych

W poniższym fragmencie występują dwa hazardy danych, na rejestrach %rsi oraz %rcx

```
add %rdi, %rsi
add %rsi, %rdx
mov (%rdi), %rcx
add %rcx, %r8
add %rdi, %r9
```

Bez zastosowania forwardingu opóźnią one sumaryczne wykonanie programu o 4 cykle, a z jego zastosowaniem opóźnią o 1 cykl.

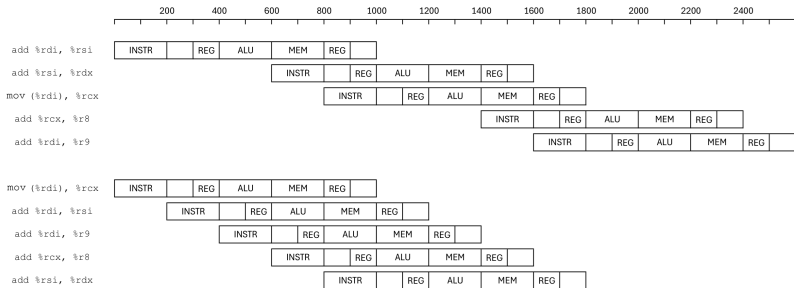
Hazardy danych

Poniżej znajduje się kod, który wykonuje dokładnie te same instrukcje, ale w innej kolejności.

```
mov (%rdi), %rcx
add %rdi, %rsi
add %rdi, %r9
add %rcx, %r8
add %rsi, %rdx
```

W powyższym kodzie nie występuje już żaden hazard.

Hazardy danych



Procesor superskalarny i out-of-order

Nowy pomysł: dołożymy do procesora kilka jednostek, które umożliwią wykonywanie kilku instrukcji jednocześnie.

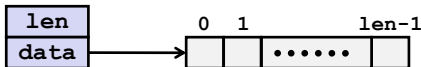
Procesor superskalarny będzie je wykonywał po kolei, out-of-order niekoniecznie.

Exploiting Instruction-Level Parallelism

- **Need general understanding of modern processor design**
 - Hardware can execute multiple instructions in parallel
- **Performance limited by data dependencies**
- **Simple transformations can cause big speedups**
 - Compilers often cannot make these transformations
 - Lack of associativity and distributivity in floating-point arithmetic

Benchmark Example: Data Type for Vectors

```
/* data structure for vectors */
typedef struct{
    size_t len;
    data_t *data;
} vec;
```



■ Data Types

- Use different declarations for `data_t`
- `int`
- `long`
- `float`
- `double`

```
/* retrieve vector element
   and store at val */
int get_vec_element
(*vec v, size_t idx, data_t *val)
{
    if (idx >= v->len)
        return 0;
    *val = v->data[idx];
    return 1;
}
```

Benchmark Computation

```
void combinel(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

Compute sum or
product of vector
elements

■ Data Types

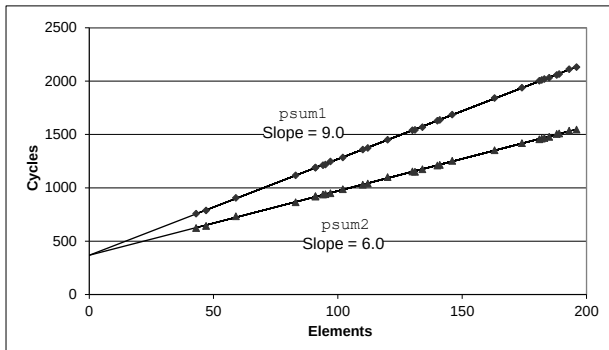
- Use different declarations for `data_t`
- `int`
- `long`
- `float`
- `double`

■ Operations

- Use different definitions of `OP` and `IDENT`
- `+` / `0`
- `*` / `1`

Cycles Per Element (CPE)

- Convenient way to express performance of program that operates on vectors or lists
- Length = n
- In our case: **CPE = cycles per OP**
- $T = \text{CPE} * n + \text{Overhead}$
 - CPE is slope of line



Benchmark Performance

```
void combine1(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

Compute sum or
product of vector
elements

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 unoptimized	22.68	20.02	19.98	20.18
Combine1 -O1	10.12	10.12	10.17	11.14
Combine1 -O3	4.5	4.5	6	7.8

Results in CPE (cycles per element)

Basic Optimizations

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

- Move `vec_length` out of loop
- Avoid bounds check on each cycle
- Accumulate in temporary

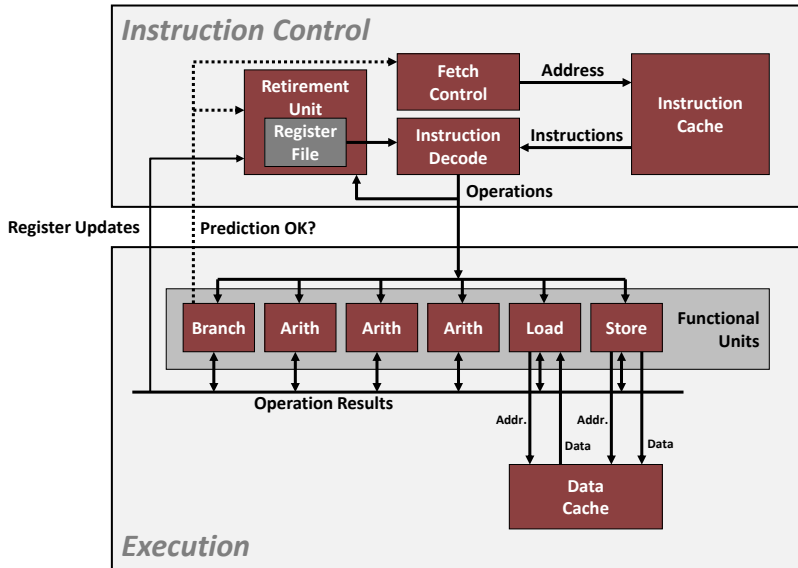
Effect of Basic Optimizations

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 -O1	10.12	10.12	10.17	11.14
Combine4	1.27	3.01	3.01	5.01

- Eliminates sources of overhead in loop

Modern CPU Design

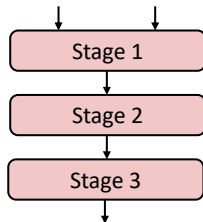


Superscalar Processor

- **Definition:** A superscalar processor can issue and execute *multiple instructions in one cycle*. The instructions are retrieved from a sequential instruction stream and are usually scheduled dynamically.
- **Benefit:** without programming effort, superscalar processor can take advantage of the *instruction level parallelism* that most programs have
- Most modern CPUs are superscalar.
- Intel: since Pentium (1993)

Pipelined Functional Units

```
long mult_eg(long a, long b, long c) {
    long p1 = a*b;
    long p2 = a*c;
    long p3 = p1 * p2;
    return p3;
}
```



Time							
	1	2	3	4	5	6	7
Stage 1	a*b	a*c			p1*p2		
Stage 2		a*b	a*c			p1*p2	
Stage 3			a*b	a*c			p1*p2

- Divide computation into stages
- Pass partial computations from stage to stage
- Stage i can start on new computation once values passed to $i+1$
- E.g., complete 3 multiplications in 7 cycles, even though each requires 3 cycles

Haswell CPU

- 8 Total Functional Units
- **Multiple instructions can execute in parallel**

2 load, with address computation
 1 store, with address computation
 4 integer
 2 FP multiply
 1 FP add
 1 FP divide

- **Some instructions take > 1 cycle, but can be pipelined**

<i>Instruction</i>	<i>Latency</i>	<i>Cycles/Issue</i>
Load / Store	4	1
Integer Multiply	3	1
Integer/Long Divide	3-30	3-30
Single/Double FP Multiply	5	1
Single/Double FP Add	3	1
Single/Double FP Divide	3-15	3-15

x86-64 Compilation of Combine4

■ Inner Loop (Case: Integer Multiply)

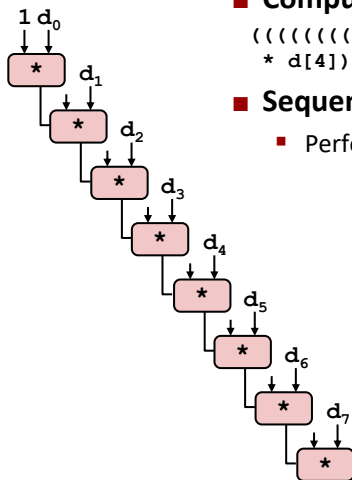
```

.L519:                                # Loop:
    imull    (%rax,%rdx,4), %ecx      # t = t * d[i]
    addq     $1, %rdx                 # i++
    cmpq     %rdx, %rbp               # Compare length:i
    jg       .L519                    # If >, goto Loop

```

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
<i>Latency Bound</i>	<i>1.00</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>

Combine4 = Serial Computation (OP = *)



■ Computation (length=8)

$(((((1 * d[0]) * d[1]) * d[2]) * d[3]) * d[4]) * d[5]) * d[6]) * d[7])$

■ Sequential dependence

- Performance: determined by latency of OP

Loop Unrolling (2x1)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = (x OP d[i]) OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- Perform 2x more useful work per iteration

Effect of Loop Unrolling

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
<i>Latency Bound</i>	<i>1.00</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>

■ Helps integer add

- Achieves latency bound

```
x = (x OP d[i]) OP d[i+1];
```

■ Others don't improve. *Why?*

- Still sequential dependency

Loop Unrolling with Reassociation (2x1a)

```
void unroll2aa_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = x OP (d[i] OP d[i+1]);
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

Compare to before

$x = (x \text{ OP } d[i]) \text{ OP } d[i+1];$

- Can this change the result of the computation?
- Yes, for FP. *Why?*

Effect of Reassociation

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
<i>Latency Bound</i>	<i>1.00</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>
<i>Throughput Bound</i>	<i>0.50</i>	<i>1.00</i>	<i>1.00</i>	<i>0.50</i>

4 func. units for int +,
2 func. units for load
Why Not .25?

1 func. unit for FP +
3-stage pipelined FP +

2 func. units for FP *,
2 func. units for load
5-stage pipelined FP *

■ Nearly 2x speedup for Int *, FP +, FP *

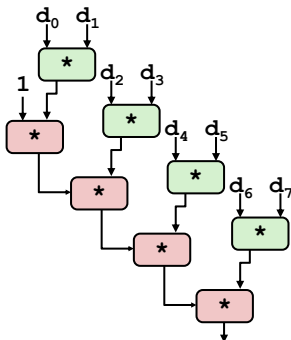
- Reason: Breaks sequential dependency

```
x = x OP (d[i] OP d[i+1]);
```

- Why is that? (next slide)

Reassociated Computation

```
x = x OP (d[i] OP d[i+1]);
```



What changed:

- Ops in the next iteration can be started early (no dependency)

Overall Performance

- N elements, D cycles latency/op
- $(N/2+1)*D$ cycles:
 $CPE = D/2$

Loop Unrolling with Separate Accumulators (2x2)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x0 = IDENT;
    data_t x1 = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x0 = x0 OP d[i];
        x1 = x1 OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x0 = x0 OP d[i];
    }
    *dest = x0 OP x1;
}
```

■ Different form of reassociation

Effect of Separate Accumulators

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
<i>Latency Bound</i>	<i>1.00</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>
<i>Throughput Bound</i>	<i>0.50</i>	<i>1.00</i>	<i>1.00</i>	<i>0.50</i>

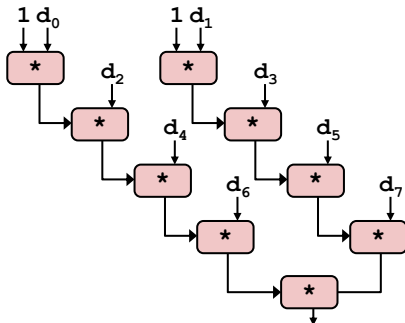
- Int + makes use of two load units

```
x0 = x0 OP d[i];
x1 = x1 OP d[i+1];
```

- 2x speedup (over unroll2) for Int *, FP +, FP *

Separate Accumulators

```
x0 = x0 OP d[i];
x1 = x1 OP d[i+1];
```



■ What changed:

- Two independent “streams” of operations

■ Overall Performance

- N elements, D cycles latency/op
- Should be $(N/2+1)*D$ cycles:
CPE = $D/2$
- CPE matches prediction!

What Now?

Unrolling & Accumulating

■ Idea

- Can unroll to any degree L
- Can accumulate K results in parallel
- L must be multiple of K

■ Limitations

- Diminishing returns
 - Cannot go beyond throughput limitations of execution units
- Large overhead for short lengths
 - Finish off iterations sequentially

Unrolling & Accumulating: Double *

■ Case

- Intel Haswell
- Double FP Multiplication
- Latency bound: 5.00. Throughput bound: 0.50

<i>Accumulators</i>	FP *	Unrolling Factor L							
	K	1	2	3	4	6	8	10	12
	1	5.01	5.01	5.01	5.01	5.01	5.01	5.01	
	2		2.51		2.51		2.51		
	3			1.67					
	4				1.25		1.26		
	6					0.84			0.88
	8						0.63		
	10							0.51	
	12								0.52

Achievable Performance

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Best	0.54	1.01	1.01	0.52
<i>Latency Bound</i>	<i>1.00</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>
<i>Throughput Bound</i>	<i>0.50</i>	<i>1.00</i>	<i>1.00</i>	<i>0.50</i>

- Limited only by throughput of functional units
- Up to 42X improvement over original, unoptimized code

Programming with AVX2

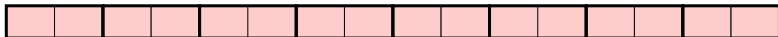
YMM Registers

■ 16 total, each 32 bytes

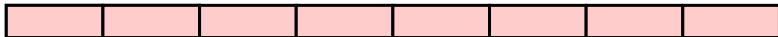
■ 32 single-byte integers



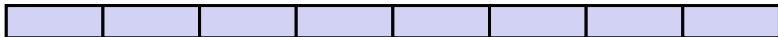
■ 16 16-bit integers



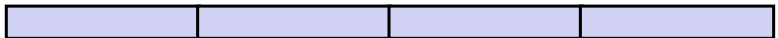
■ 8 32-bit integers



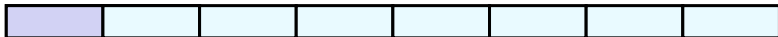
■ 8 single-precision floats



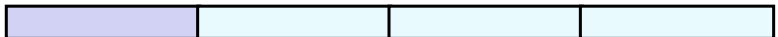
■ 4 double-precision floats



■ 1 single-precision float



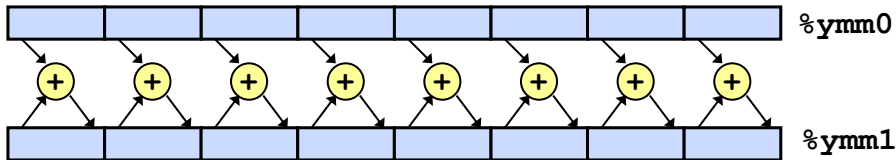
■ 1 double-precision float



SIMD Operations

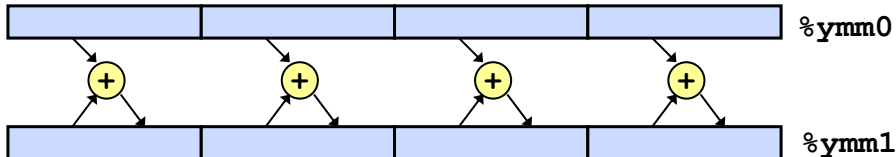
■ SIMD Operations: Single Precision

vaddps %ymm0, %ymm1, %ymm1



■ SIMD Operations: Double Precision

vaddpd %ymm0, %ymm1, %ymm1



Using Vector Instructions

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Scalar Best	0.54	1.01	1.01	0.52
Vector Best	0.06	0.24	0.25	0.16
<i>Latency Bound</i>	<i>0.50</i>	<i>3.00</i>	<i>3.00</i>	<i>5.00</i>
<i>Throughput Bound</i>	<i>0.50</i>	<i>1.00</i>	<i>1.00</i>	<i>0.50</i>
<i>Vec Throughput Bound</i>	<i>0.06</i>	<i>0.12</i>	<i>0.25</i>	<i>0.12</i>

■ Make use of AVX Instructions

- Parallel operations on multiple data elements
- See Web Aside OPT:SIMD on CS:APP web page

Procesor superskalarny i out-of-order

Do symulacji wykonania programu na wybranej mikroarchitekturze (np. Haswell) można użyć narzędzia `llvm-mca`, na przykład komendą:

```
llvm-mca -mcpu=haswell -timeline -iterations=5 sum.s
```

Więcej szczegółów na temat narzędzia albo charakterystyki wypisanego schematu można znaleźć na przykład tutaj:
<https://llvm.org/docs/CommandGuide/llvm-mca.html>.

Charakterystykę jednostek funkcyjnych w mikroarchitekturze Haswell można zobaczyć na przykład tutaj:
<https://www.agner.org/optimize/microarchitecture.pdf>, na stronie 142.